Courbe de Koch

Michaël YANG

Sorbonne Université

18 mars 2026

Plan

1. Pourquoi cette Fractale ?
2. Quel IFS choisir pour la Courbe de Koch ?
   - Pourquoi l'IFS permet d'obtenir la Fractale
3. Algorithme pour représenter la Courbe de Koch
4. Le code et l'affichage

Pourquoi cette Fractale

- Déjà rencontré auparavant
- Facile à vu d'oeil (des segments)
- Belle graphiquement (+ variantes)

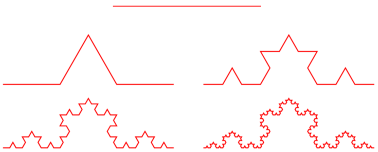


Figure – Courbe de Koch

Source : *Understanding Fractality : A Polyhedral Approach to the Koch Curve and its Complex Dimensions*, David Claire Lapidus Michel L, 2023

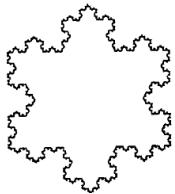


Figure – Flocon de Koch

Source : *Fractal geometry : mathematical foundations and applications*, Kenneth Falconer,..., 2014

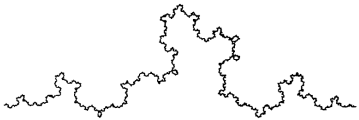


Figure – Koch aléatoire

Source : *Fractal geometry : mathematical foundations and applications*, Kenneth Falconer,..., 2014

Quel IFS choisir pour la Courbe de Koch ?

Le Principe :

- transformer un segment en 4 autres
- tranformer les 4 segments en 16 autres
- ...

Il nous faut donc les coordonées de chaques nouveaux points après transformation

Quel IFS choisir pour la Courbe de Koch ?

-Il nous faut la Matrice de rotation : $\mathcal{R}_\theta = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$

-Utilisée par 4 applications qui "transforment" les positions des points initiales

-On note notre IFS :

$$S = \{S_1, S_2, S_3, S_4\}$$

$$u = \begin{pmatrix} x \\ y \end{pmatrix} \qquad S_i : \mathbb{R}^2 \to \mathbb{R}^2$$

$$S_1(u) = \frac{1}{3}\begin{pmatrix} x \\ y \end{pmatrix} \qquad S_2(u) = \frac{1}{3}\mathcal{R}_{\pi/3}\begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} \frac{1}{3} \\ 0 \end{pmatrix}$$

$$S_3(u) = \frac{1}{3}\mathcal{R}_{-\pi/3}\begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} \frac{1}{2} \\ \frac{\sqrt{3}}{6} \end{pmatrix} \qquad S_4(u) = \frac{1}{3}\begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} \frac{2}{3} \\ 0 \end{pmatrix}$$

Pourquoi l'IFS permet d'obtenir la Fractale

S est composée d'applications contractantes :

- $\mathbb{R}^2$ est un espace métrique
- Les S_i sont des applications affines du type $ax + b$
- en prenant $u, v \in \mathbb{R}^2$:

$$S_i(u) - S_i(v) = au + b - (av + b) = a(u - v)$$

En prenant ensuite la norme :

$$\|S_i(u) - S_i(v)\| = \|a(u - v)\| = |a|\|u - v\|$$

Or ici on a que $a = \frac{1}{3} \in [0, 1[$:

$$\|S_i(u) - S_i(v)\| = |\frac{1}{3}|\|u - v\| \leq |\frac{1}{3}|\|u - v\|$$

Pourquoi l'IFS permet d'obtenir la Fractale

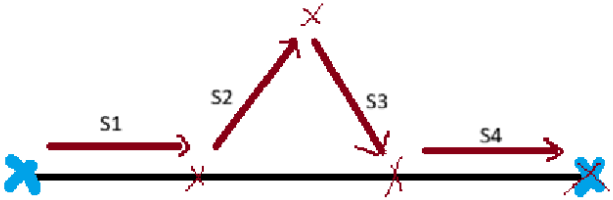
- On définit alors l'opérateur de Hutchinson :

$$\mathscr{S}(A) = S_1(A) \cup S_2(A) \cup S_3(A) \cup S_4(A), \qquad A \in \mathcal{A}.$$

- Comme il est composé de contractions, $\mathscr{S}$ est lui aussi contractant (pour la distance de Hausdorff)
- Par le théorème du point fixe de Banach, il existe un unique ensemble K tel que $\mathscr{S}(K) = K$
- Cet ensemble K est appelé **l'attracteur de l'IFS** (= la Fractale)

Algorithme pour représenter la Courbe de Koch

- Construire des segments à partir d'un segment
- Prendre les coordonnées d'un point et en renvoyer 4 autres
- réitérer sur ces 4 nouveaux points...



Le Code et l'affichage

Matrice de rotation :

```
def rotation(dx: float, dy: float, theta: float) -> Tuple[float, float]:
    xr: float = dx * math.cos(theta) - dy * math.sin(theta)
    yr: float = dx * math.sin(theta) + dy * math.cos(theta)
    return (xr, yr)
```

Le Code et l'affichage

signature de la fonction récursive et cas d'arrêt :

```
def koch_segment(x1: float, y1: float, x2: float, y2: float, n: int) -> Image:
    if n == 0:
        return draw_line(x1, y1, x2, y2)
```

Le Code et l'affichage

vecteurs de translations/rotations :

```
dx: float = x2 - x1
dy: float = y2 - y1

v1x: float = dx/3
v1y: float = dy/3

v4x: float = dx/3
v4y: float = dy/3

rdx, rdy = rotation(dx/3, dy/3, math.pi/3)
v2x: float = rdx
v2y: float = rdy

rdx2, rdy2 = rotation(dx/3, dy/3, -math.pi/3)
v3x: float = rdx2
v3y: float = rdy2
```

Le Code et l'affichage

Coordonnées des nouveaux points

```
ax: float = x1
ay: float = y1

bx: float = ax + v1x
by: float = ay + v1y

cx: float = bx + v2x
cy: float = by + v2y

dx: float = cx + v3x
dy: float = cy + v3y

#ex: float = dx + v4x
#ey: float = dy + v4y
```

Le Code et l'affichage

Récursion sur ces points et retour :

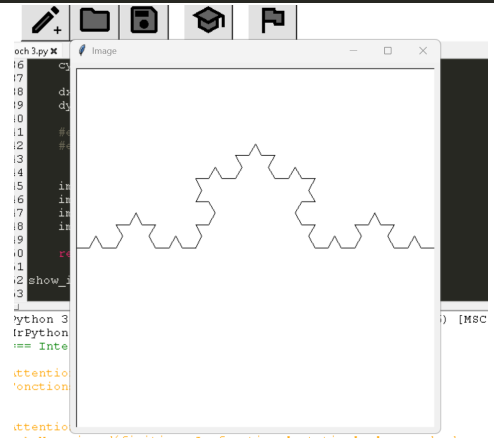
```
img1: Image = koch_segment(ax, ay, bx, by, n - 1)
img2: Image = koch_segment(bx, by, cx, cy, n - 1)
img3: Image = koch_segment(cx, cy, dx, dy, n - 1)
img4: Image = koch_segment(dx, dy, x2, y2, n - 1)

return overlay(img1, img2, img3, img4)
```

Le Code et l'affichage

Affichage pour n = 3 :

```
show_image(koch_segment(-1.0, 0.0, 1.0, 0.0, 3))
```



Références I

Claire David and Michel L Lapidus.
Understanding fractality : A polyhedral approach to the koch curve and its complex dimensions.
Asymptotic analysis, 2023.

Kenneth J. Falconer.
Fractal geometry : mathematical foundations and applications / Kenneth Falconer,...
Wiley, Chichester (GB), 3rd ed. edition, 2014.

John E. Hutchinson.
Fractals and self-similarity.
Indiana Univ. Math. J., 30(5) :713–747, 1981.

Microsoft.
Microsoft copilot.
https://www.microsoft.com/copilot, 2024.
Assistant IA utilisé pour l'aide à la rédaction et l'explication de concepts mathématiques.

Wikipédia.
Distance de hausdorff — wikipédia, l'encyclopédie libre, 2025.
[En ligne ; Page disponible le 28-septembre-2025], vu le 17/03/2026.